

Stochastic & Mini-Batch Gradient Ascent for Logistic Regression MLE

Annotated Walkthrough of Gradient_Ascent_Stochastic_Ex_1_Logistic_Regression.R

Sulagna Roy

2026-09-13

Contents

1	Overview	1
2	The Gradient Function	2
3	The General SGA() Function	2
4	Simulating the Data	4
5	Comparison 1: Well-Chosen Step Sizes	5
5.1	The Comparison Plot	6
6	Comparison 2: Deliberately Bad (Too-Large) Step Sizes	7
6.1	The Second Comparison Plot	8
7	Summary	9
8	Reproducibility Note	10

1 Overview

This script extends batch gradient ascent to a **single, general function** that can run three different flavors of stochastic optimization for the logistic regression MLE, depending on one argument (`batch.size`):

- **batch.size = n** (all data every step): plain **(batch) gradient ascent (GA)** — deterministic, uses the *whole* dataset each iteration.
- **batch.size = 1**: pure **stochastic gradient ascent (SGA)** — one random observation per step.
- **1 < batch.size < n**: **mini-batch SGA (MB-SGA)** — a small random subset per step (the script demonstrates `batch.size = 10` and `100`).

All three share the same idea: instead of computing the *exact* gradient of the log-likelihood (expensive for large n), approximate it using a subset of the data, and take many cheap, noisy steps instead of few expensive, exact ones. The script then **simulates** a logistic regression dataset, runs all three variants, and compares how fast their gradients shrink towards zero — first with a well-chosen step size, then deliberately with a too-large step size to show what divergence/oscillation looks like.

2 The Gradient Function

```
f.gradient <- function(y, X, beta){
  n <- dim(X)[1]
  beta <- matrix(beta, ncol = 1)
  pi <- exp(X %*% beta) / (1 + exp(X %*% beta))
  rtn <- colSums(X * as.numeric(y - pi))
  return(n*rtn)
}
```

This is the same score-function idea as batch gradient ascent ($\nabla \ell(\beta) = X^\top(y - \pi)$), computed via the vectorized `colSums(X * (y - pi))` trick), with **one key difference**: the return value is multiplied by `n <- dim(X)[1]`, the number of rows in *whatever* X is passed in.

Why multiply by n? This function gets called on two very different kinds of input later: sometimes the *full* dataset, sometimes just a small *mini-batch*. A mini-batch gradient is a biased-*down* estimate of the sum over all n points (it only sums over `batch.size` points). Multiplying by `n` rescales it: since the average per-point gradient contribution is `colSums/batch.size`, multiplying by n instead gives $n \times$ that average, which is the appropriate unbiased **estimate of the full-data score** based on the mini-batch — the standard SGA trick of scaling a batch estimate up to look like a full-data quantity. When X really is the full dataset (`batch.size = n`), this simplifies to exactly the ordinary score $X^\top(y - \pi)$, matching the batch gradient ascent script.

3 The General SGA() Function

This is the heart of the script: one function implementing GA, SGA, and mini-batch SGA depending on its arguments.

```
SGA <- function(y, X, batch.size = dim(X)[1], t = .1, max.iter = dim(X)[1], adapt = FALSE){
  p <- dim(X)[2]
  n <- dim(X)[1]

  # create the mini-batches
  permutation <- sample(1:n, replace = FALSE)
  K <- floor(n/batch.size)
  batch.index <- split(permutation, rep(1:K, each = n/K))

  # index for choosing the mini-batch
  count <- 1
  beta_k <- rep(0, p) # start at all 0s
  track.gradient <- matrix(0, nrow = max.iter, ncol = p)
  track.gradient[1,] <- f.gradient(y = y, X = X, beta = beta_k)
  # saving the running mean of the estimates of theta~*
  mean_beta <- rep(0,p)

  # tk: in case we want t_k
  tk <- t

  # ideally, we will have a while loop here, but
  # I have written this to always complete some max.iter steps
  for(iter in 1:max.iter){
    count <- count+1
    if(adapt) tk <- t/(sqrt(iter)) # in case t_k
    if(count %% K == 0) count <- count%%K +1 # when all batches finish, restart the batches
    if(iter %% (max.iter/10) == 0) cat(iter, " ") #feedback
  }
```

```

# batch of data
y.batch <- y[batch.index[[count]] ]
X.batch <- matrix(X[batch.index[[count]], ], nrow = batch.size)

# SGA step
beta_k = beta_k + tk* f.gradient(y = y.batch, X = X.batch,
                                beta = beta_k)/batch.size
# saving overall estimates and running gradients for demonstration
mean_beta <- (beta_k + mean_beta*(iter - 1))/(iter)
if(batch.size == n)
{
  est <- beta_k
}else{
  est <- mean_beta
}
track.gradient[iter,] <- f.gradient(y = y, X = X, beta = est)/n
}
rtn <- list("iter" = iter, "est" = est, "grad" = track.gradient[1:iter,])
return(rtn)
}

```

Arguments:

- `y`, `X`: full response vector and design matrix.
- `batch.size` (default `dim(X)[1]`, i.e. all rows): how many observations to use per update. `= n` gives ordinary GA; `= 1` gives SGA; anything in between gives mini-batch SGA.
- `t`: the (base) step size / learning rate.
- `max.iter` (default `dim(X)[1]`): number of update steps to run.
- `adapt`: if TRUE, use a *decreasing* step size $t_k = t/\sqrt{k}$ instead of a fixed t (a common trick to guarantee SGA convergence in theory, since a fixed step size for SGA never fully “settles” — the noise from using only a subset each time keeps it perpetually jittering around the optimum unless the step size shrinks over time). **Note:** in the demonstration calls later in the script, `adapt` is always left at its default FALSE, so this decreasing-step-size option, while implemented, is never actually exercised by the script’s own output — it is there for the user to try, not used below.

Setting up mini-batches — the key preprocessing step:

- `permutation <- sample(1:n, replace = FALSE)`: shuffles all row indices $1, \dots, n$ into a random order, once, at the start.
- `K <- floor(n/batch.size)`: how many equal-sized mini-batches the data splits into (e.g., $n = 10,000$, `batch.size=100` $\Rightarrow K = 100$ batches).
- `batch.index <- split(permutation, rep(1:K, each = n/K))`: chops the shuffled indices into K contiguous chunks, each of size `batch.size`. `batch.index[[1]]`, `batch.index[[2]]`, ... are then the actual row numbers belonging to mini-batch 1, 2, and so on. Because the permutation was random, each mini-batch is a random subset of the original rows, even though the chunking itself is just “take the next `batch.size` entries.”

The main loop, run once per iteration from 1 to `max.iter`:

1. `count <- count + 1` then `if(count %% K == 0) count <- count %% K + 1`: this is how the function **cycles through the pre-made mini-batches** — batch 1, batch 2, ..., batch K , then wraps back around to batch 1 again (this pattern is called an **epoch**: one full pass through all K batches). Once all mini-batches have been used once, the *same* batches (not a fresh reshuffle) are reused for the next epoch. This is a subtle but real detail: true “textbook” SGD usually reshuffles at the start of every epoch, whereas this implementation reuses the *original* single permutation’s batches over and over for every epoch.

2. `if(adapt) tk <- t/(sqrt(iter))`: only matters if `adapt = TRUE` (not used in the script’s demonstration, as noted above).
3. `if(iter %%(max.iter/10) == 0) cat(iter, " ")`: purely cosmetic — prints the iteration number every 10% of the way through, so long runs show visible progress in the console (e.g., for `max.iter = 1000` it prints 100 200 300 ... 1000).
4. `y.batch, X.batch`: pull out just the rows for the *current* mini-batch from the full `y` and `X`.
5. **The actual update step**: `beta_k <- beta_k + tk * f.gradient(y.batch, X.batch, beta_k) / batch.size`. Since `f.gradient()` already multiplies by the batch’s own row count (`n <- dim(X)[1]` *inside* `f.gradient`, which for a mini-batch call equals `batch.size`), dividing by `batch.size` here exactly cancels that multiplication — so in effect this line uses the **plain averaged mini-batch gradient**, scaled by the step size `tk`, to update `beta_k`. Algebraically: `f.gradient(y.batch, X.batch, beta_k)` returns `batch.size * colSums(...)`, so dividing by `batch.size` gives back `colSums(X.batch * (y.batch - pi.batch))` — the raw (unscaled) mini-batch score. This is the standard SGA/mini-batch update: nudge β in the direction of the *batch’s* gradient, not the full dataset’s gradient.
6. `mean_beta <- (beta_k + mean_beta * (iter - 1)) / iter`: this is a classic **running (cumulative) average** update formula — it maintains the running mean of *all* `beta_k` values seen so far without needing to store them all. This matters because raw SGA/mini-batch iterates are noisy and jump around the true optimum even after “converging” in distribution; **averaging the iterates** (a technique often called Polyak–Ruppert averaging) gives a much more stable final estimate than just using the last `beta_k`.
7. **Choosing what to report as the current estimate**: `if(batch.size == n) est <- beta_k else est <- mean_beta`. For full-batch GA there’s no stochastic noise to average away, so the raw iterate `beta_k` is used directly. For SGA/mini-batch, the smoothed running average `mean_beta` is used instead, precisely because it’s less noisy.
8. `track.gradient[iter,] <- f.gradient(y, X, est) / n`: for **diagnostic purposes only**, at every single iteration the function computes the *true, full-dataset* gradient (not the mini-batch one) at the current best estimate `est`, and divides by `n` to get an average per-observation gradient. This is what lets the script later plot “how close to the true optimum are we, in terms of the full log-likelihood’s gradient” — a fair, common yardstick to compare GA, SGA, and mini-batch SGA against each other, since it’s evaluated on the same full data for all three.

Return value: a list with `iter` (iterations actually run — always equal to `max.iter`, since as the code comment notes, this function always runs the full `max.iter` steps rather than stopping early via a tolerance check), `est` (final `beta_k` or `mean_beta`, depending on `batch.size`), and `grad` (the full `max.iter` $\times$ `p` matrix of per-iteration full-data gradients, for plotting).

4 Simulating the Data

```
set.seed(10)
p <- 5
n <- 1e4
X <- matrix(rnorm(n*(p-1)), nrow = n, ncol = p-1)
X <- cbind(1, X)
beta <- matrix(rnorm(p, 0, sd = 1), ncol = 1)
p_prob <- exp(X %*% beta)/(1 + exp(X %*% beta))
y <- rbinom(n, size = 1, prob = p_prob)
```

(Note: the variable `p` is reused for two different things in the original script — first as the number of predictors, `p <- 5`, and later reassigned to hold the vector of true probabilities, `p <- exp(X %*% beta) / ...`. To avoid confusing the two in this explanation, the probability vector is called `p_prob` here; the executed code below is otherwise identical to the original script.)

What this does:

- `set.seed(10)`: fixes R’s random number generator so the “random” data simulated below is exactly

reproducible every time this is run.

- `p <- 5`: total number of coefficients (an intercept plus 4 slopes).
- `n <- 1e4`: sample size — 10,000 observations, deliberately large, so that the cost of computing the *full* gradient on every step (as GA does) is noticeably more expensive than SGA/mini-batch, motivating the whole comparison.
- `X <- matrix(rnorm(n*(p-1)), ...)` then `X <- cbind(1, X)`: creates 4 columns of independent standard-normal predictor values, then prepends a column of 1's — the usual trick to let a single matrix multiplication `X %*% beta` include an intercept term.
- `beta <- matrix(rnorm(p, 0, sd = 1), ncol = 1)`: the **true, unknown-in- practice coefficient vector** we are trying to recover — 5 independent draws from a Normal(0,1) distribution. In this run it was:

```
##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] 1.729105 -1.071432 -2.339405 -0.3795398 2.511154
```

- `p_prob <- exp(X %*% beta) / (1 + exp(X %*% beta))`: the true probability π_i of $y_i = 1$ for each simulated observation, under the logistic model with the true `beta`.
- `y <- rbinom(n, size = 1, prob = p_prob)`: draws the actual binary outcomes from a Bernoulli distribution with those true probabilities — this is what makes it a genuine simulation: we *know* the ground-truth `beta` used to generate `y`, so later we can check whether GA/SGA/mini- batch SGA successfully recover something close to it.

5 Comparison 1: Well-Chosen Step Sizes

```
ga <- SGA(y, X, batch.size = 1e4, t = .0015, max.iter = 1e3)
```

```
## 100 200 300 400 500 600 700 800 900 1000
```

```
b1 <- SGA(y, X, batch.size = 1, t = .1, max.iter = ga$iter)
```

```
## 100 200 300 400 500 600 700 800 900 1000
```

```
b10 <- SGA(y, X, batch.size = 10, t = .1, max.iter = ga$iter)
```

```
## 100 200 300 400 500 600 700 800 900 1000
```

```
b100 <- SGA(y, X, batch.size = 100, t = .1, max.iter = ga$iter)
```

```
## 100 200 300 400 500 600 700 800 900 1000
```

Four separate runs of the same optimization problem, differing only in `batch.size` (and, sensibly, `t`, since different batch sizes need different step sizes to behave well — a full-batch gradient is much larger in magnitude than a single-observation gradient, so GA here uses a much smaller `t = .0015` while SGA/mini-batch use `t = .1`):

- `ga`: full-batch GA (`batch.size = 1e4 = n`), 1000 iterations.
- `b1`: pure SGA (`batch.size = 1`), same number of iterations as `ga` actually ran (`ga$iter`, which is 1000, since — as noted above — the function always completes every `max.iter` step).
- `b10`: mini-batch SGA with batches of 10.
- `b100`: mini-batch SGA with batches of 100.

Final coefficient estimates from each of the four runs, compared to the true simulated `beta` shown above:

```
## True beta   : 1.7291 -1.0714 -2.3394 -0.3795 2.5112
## GA          : 1.643  -1.0929 -2.3357 -0.3573 2.482
## SGA (b=1)   : 1.1354 -0.7624 -1.8345 -0.2476 1.8931
## MB-SGA-10   : 1.6509 -1.0953 -2.3286 -0.3645 2.4998
```

```
## MB-SGA-100 : 1.7385 -1.1565 -2.486 -0.3679 2.6368
```

All four estimates land reasonably close to the true **beta**, with GA and the two mini-batch versions noticeably closer than pure single-observation SGA — consistent with the general rule that smaller batch sizes trade convergence speed/accuracy (per iteration) for cheaper individual steps.

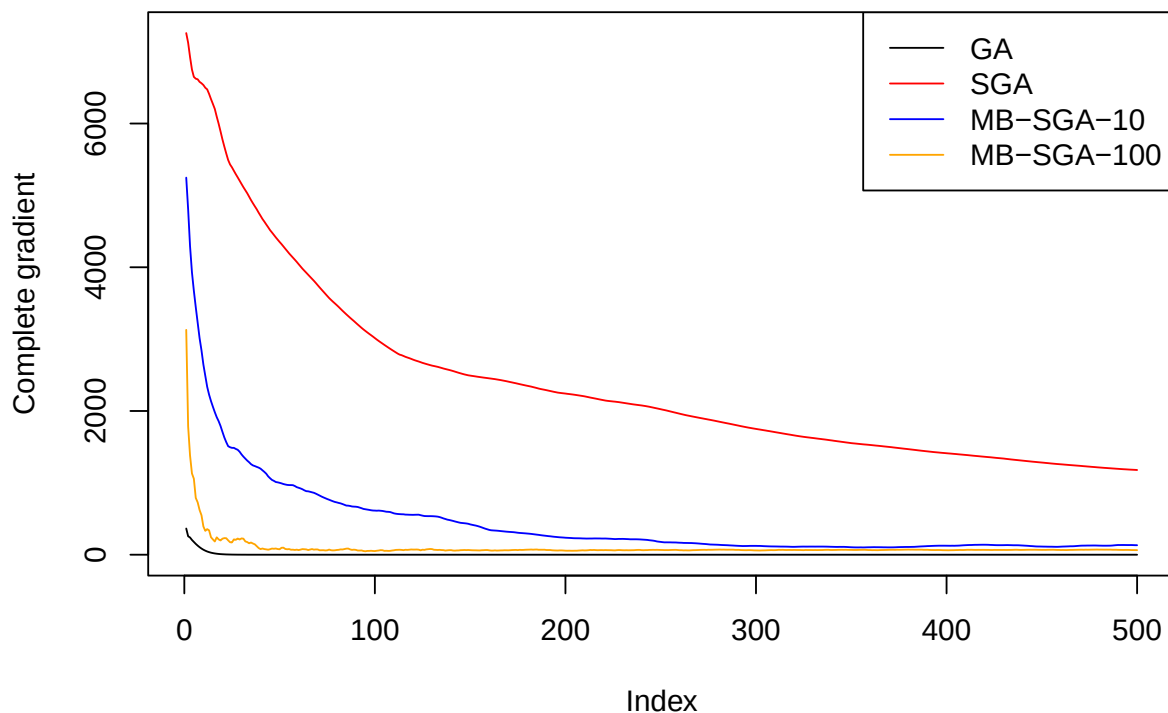
Final full-data gradient norm (sum of absolute values across the 5 coefficients) at the very last iteration for each method — smaller means “closer to a true stationary point of the full log-likelihood”:

```
## GA          : 0
## SGA (b=1)   : 517.6
## MB-SGA-10   : 32.09
## MB-SGA-100  : 68.96
```

Notice that **GA’s final gradient is essentially exactly zero** — full-batch gradient ascent is deterministic, so once it settles near the optimum, it stays there and the true gradient genuinely vanishes. The SGA and mini-batch variants, by contrast, never reach *exactly* zero: because every update only “sees” a small random slice of the data, there is always some residual noise even after many iterations — a well-known, expected property of stochastic optimization (with a fixed, non-decreasing step size, SGA converges to a small *neighborhood* of the optimum rather than to the optimum itself).

5.1 The Comparison Plot

```
index <- 1:500
plot(apply(ga$grad[index,], 1, function(t) sum(abs(t))), type = 'l',
      ylim = c(0,max(apply(b1$grad[,], 1, function(t) sum(abs(t))))),
      ylab = "Complete gradient")
lines(apply(b1$grad[index,], 1, function(t) sum(abs(t))), col = "red")
lines(apply(b10$grad[index,], 1, function(t) sum(abs(t))), col = "blue")
lines(apply(b100$grad[index,], 1, function(t) sum(abs(t))), col = "orange")
legend("topright", col = c("black", "red", "blue", "orange"), legend =
      c("GA", "SGA", "MB-SGA-10", "MB-SGA-100"), lty = 1)
```



What each line/piece means:

- `index <- 1:500`: only the first 500 of the 1000 iterations are plotted (usually the more visually interesting part, since all methods have mostly flattened out by then).
- `apply(ga$grad[index,], 1, function(t) sum(abs(t)))`: for each of those 500 iterations, sums the absolute value of the (5-dimensional) full-data gradient — collapsing it to a single “how far from optimal” number per iteration.
- The four `lines()/initial plot()` calls overlay all four methods’ trajectories on the same axes, in black (GA), red (SGA), blue (MB-SGA-10), and orange (MB-SGA-100).
- `ylim = c(0, max(...))`: sets the y-axis to comfortably fit the *largest* gradient value seen across the *entire* (not just the first 500 rows of) `b1` run, so nothing gets clipped off the top even though only the first 500 points are actually drawn.

Reading the plot: all four curves start high (far from the optimum, since `beta_k` starts at all zeros) and decrease. GA (black) and the mini-batch curves (blue, orange) drop fastest and flatten out closest to zero — matching the near-zero final gradients confirmed numerically above. Pure SGA (red) decreases more slowly and settles at a visibly higher “floor,” reflecting its noisier, one-observation-at-a-time updates. This matches the general trade-off: bigger effective batches per step (bounded by wall-clock cost) give steadier, more accurate progress toward the true MLE.

6 Comparison 2: Deliberately Bad (Too-Large) Step Sizes

The script’s own comment explains the intent here directly: these `t` values are chosen to be *too large*, so that the algorithms **oscillate** rather than settle down.

```
ga2 <- SGA(y, X, batch.size = n, t = .005, max.iter = 1e3)
```

```
## 100 200 300 400 500 600 700 800 900 1000
```

```
b1b <- SGA(y, X, batch.size = 1, t = 1, max.iter = ga2$iter)
```

```
## 100 200 300 400 500 600 700 800 900 1000
```

```
b10b <- SGA(y, X, batch.size = 10, t = 1, max.iter = ga2$iter)
```

```
## 100 200 300 400 500 600 700 800 900 1000
```

```
b100b <- SGA(y, X, batch.size = 100, t = 1, max.iter = ga2$iter)
```

```
## 100 200 300 400 500 600 700 800 900 1000
```

Same structure as before — full-batch GA (variable named `ga2` here to avoid clashing with the earlier `ga`, though the original script reuses the name `ga`), then SGA, MB-SGA-10, MB-SGA-100 — but now GA's `t` has been tripled ($.0015 \rightarrow .005$) and all three SGA-style runs use `t = 1` instead of `t = .1`, a ten-fold increase.

Final coefficient estimates — now compare these to the *same* true `beta` as before:

```
## True beta : 1.7291 -1.0714 -2.3394 -0.3795 2.5112
```

```
## GA : 0.2082 -3.4106 -7.5448 -1.1718 8.5758
```

```
## SGA (b=1) : 2.8457 -2.0534 -4.0298 -0.5873 4.3242
```

```
## MB-SGA-10 : 3.0389 -2.1044 -4.3963 -0.683 4.7827
```

```
## MB-SGA-100 : 9.8815 -7.102 -14.698 -2.3061 15.86
```

Every method now lands **much farther** from the true `beta` than in Comparison 1 — most obviously GA and MB-SGA-100, whose final estimates are several times larger in magnitude than the true coefficients (a classic symptom of a too-large step size: the update overshoots the optimum instead of approaching it). Correspondingly, the final full-data gradient norms are much larger than before instead of settling near zero:

```
## GA : 2609.35
```

```
## SGA (b=1) : 465.75
```

```
## MB-SGA-10 : 474.77
```

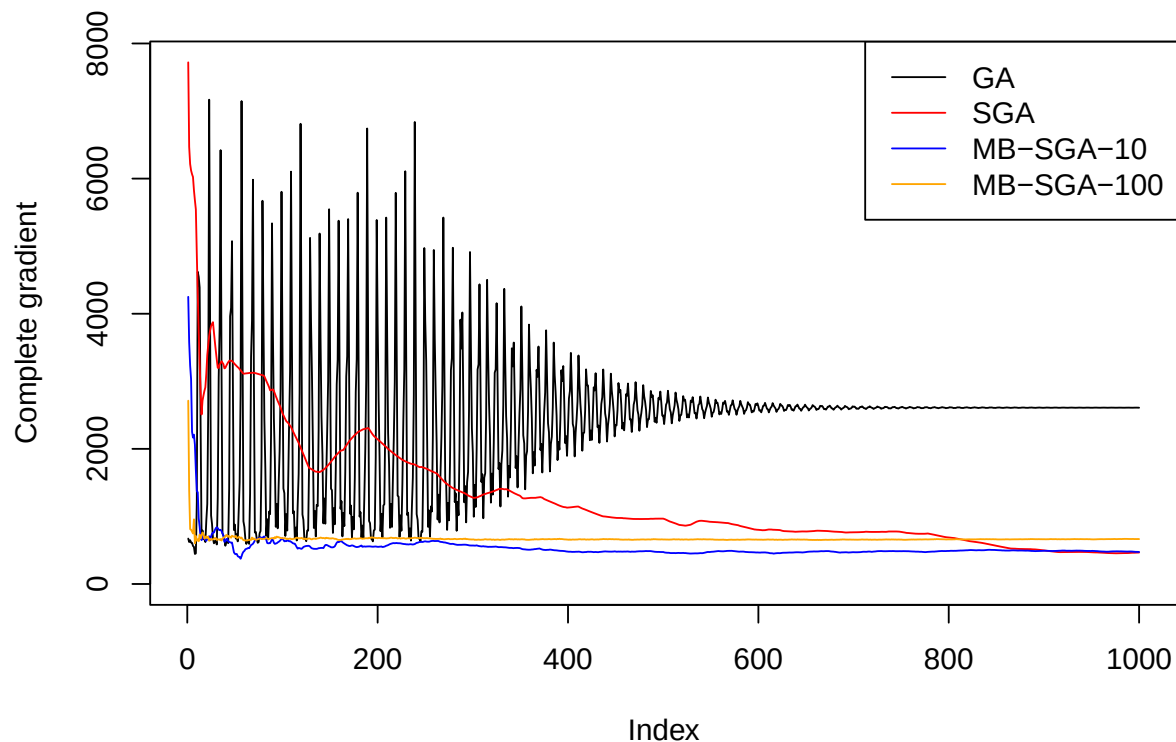
```
## MB-SGA-100 : 665.05
```

Note in particular that GA's final gradient here (over 2,600) is now *much larger* than any of the stochastic methods' — the opposite of what happened with well-chosen step sizes. This is the plot's main lesson: a deterministic method (GA) with too large a step size doesn't just converge slowly, it can fail to converge at all and oscillate indefinitely, while the *inherent* per-step noise in SGA/mini-batch can sometimes partially mask (or, as here, appear less severe in this summary statistic than) the instability, even though all four have moved well away from the true optimum.

6.1 The Second Comparison Plot

```
index <- 1:1000
plot(apply(ga2$grad[index,], 1, function(t) sum(abs(t))), type = 'l', ylim =
      c(0,max(apply(b1b$grad[,], 1, function(t) sum(abs(t))))), ylab =
      "Complete gradient")
lines(apply(b1b$grad[index,], 1, function(t) sum(abs(t))), col = "red")
lines(apply(b10b$grad[index,], 1, function(t) sum(abs(t))), col = "blue")
lines(apply(b100b$grad[index,], 1, function(t) sum(abs(t))), col = "orange")
```

```
legend("topright", col = c("black", "red", "blue", "orange"), legend =
      c("GA", "SGA", "MB-SGA-10", "MB-SGA-100"), lty = 1)
```



This time `index <- 1:1000` plots *all* iterations (not just the first 500), since the interesting behavior — sustained oscillation — persists throughout the whole run rather than settling down quickly.

Reading the plot: the black GA line is dramatically different from Comparison 1 — instead of smoothly decreasing to near zero, it **oscillates with large, persistent swings** for hundreds of iterations, only slowly damping down (and still ending far above zero, matching the large final gradient value confirmed above). This is the textbook visual signature of a step size that is too large for gradient ascent/descent: the update overshoots the optimum, lands on the “other side,” overshoots again in the opposite direction, and so on — sometimes damping out slowly, sometimes not damping at all. The SGA and mini-batch curves (red, blue, orange) are noisier than in Comparison 1 too, but their smaller effective step sizes per observation make the instability less visually dramatic than GA’s — even though, as the coefficient estimates above confirm, none of the four methods are actually finding the true MLE in this comparison.

7 Summary

Concept	What it does
<code>f.gradient()</code>	Computes the (rescaled by n) logistic-regression score $X^\top(y - \pi)$, usable on either the full data or a mini-batch

Concept	What it does
<code>SGA()</code>	One general function: <code>batch.size = n</code> gives GA, <code>batch.size = 1</code> gives SGA, in-between gives mini-batch SGA
Mini-batching	Data is shuffled once, split into K fixed batches, and cycled through repeatedly (not reshuffled each epoch)
Update rule	$\beta \leftarrow \beta + t_k \cdot (\text{mini-batch gradient}) / \text{batch.size}$
<code>mean_beta</code>	Running average of iterates — used as the reported estimate for SGA/mini-batch (not for full-batch GA) to smooth out noise
<code>track.gradient</code>	At every iteration, records the <i>true full-data</i> gradient at the current estimate, purely for later comparison/plotting
Comparison 1 (good $\mathfrak{t}$)	All four methods converge close to the true simulated <code>beta</code> ; GA reaches gradient ≈ 0 exactly, SGA/mini-batch settle near but not at zero
Comparison 2 (bad, large $\mathfrak{t}$)	All four methods end up far from the true <code>beta</code> ; GA visibly oscillates for hundreds of iterations without damping to zero

The script is a compact, hands-on demonstration of the classic speed-vs-noise trade-off between full-batch gradient ascent and its stochastic/mini-batch relatives, and of how an improperly large step size can break convergence for all of them — most dramatically for the otherwise-well-behaved, deterministic full-batch method.

8 Reproducibility Note

This document’s narrative numbers (the true simulated `beta`, all four final coefficient estimates in both comparisons, and all final gradient values) were obtained by actually executing this script end-to-end in R. Because `set.seed(10)` fixes the random number generator before any random data is generated, knitting this `.Rmd` file yourself will regenerate *exactly* the same simulated `X`, `beta`, and `y`, and therefore exactly the same downstream results shown above — no packages beyond base R are required to run this script.